ITAI2372- Module06- AI in Retail – Stuart Fairchild

I chose a locally hosted Ollama model (`llama3.2:3b` by default) rather than a hosted LLM API because it makes the prototype inexpensive to run after the initial model download, avoids sending product inputs to an external model provider, and works within a Docker Compose environment. The smaller model is a practical starting point for a classroom demonstration; the model can be changed through the `OLLAMA_MODEL` environment variable. The application includes an explainable customer-segment rule layer: for example, budget- conscious shoppers receive value-oriented guidance, while premium buyers receive more restrained design-oriented guidance. Tone selection is separate from segment selection, allowing the same product facts to be adapted for different audiences without changing the underlying product claims.

The main implementation challenges are coordinating startup between the Ollama server, model download, and Flask application; handling the first model download and slower generation times; and writing prompts that produce persuasive copy without inventing specifications or unsupported claims. The Compose setup addresses startup ordering with a model-initialization service and persistent model storage. Future extensions could connect the app to a verified product information management (PIM) catalog, generate and compare multiple variants, support localization and accessibility checks, measure conversion outcomes through controlled experiments, and add brand-voice templates with human approval. In a real retail setting, personalization should use relevant and consented data, avoid sensitive-trait inference and manipulative targeting, and test for biased or exclusionary language. Product claims should be traceable to approved catalog facts, and human review should remain part of the publishing workflow.